JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

Introduction to Generative AI

Advanced GAN Methods and Applications

Outline

- Module Learning Objectives
- Generative Adversarial Networks and their Optimization
- Challenges in GAN Training and their Solutions
- Various GANs

Module Learning Objectives

- Understand key principles of advanced GAN architectures, such as Wasserstein GAN, CycleGAN, and StyleGAN
- Identify and mitigate issues like mode collapse, vanishing gradients, and instability in GAN training
- Compare divergence measures like Jensen-Shannon and Wasserstein distances and their impact on GAN optimization
- Examine the utility of conditional GANs, InfoGAN, and other specialized variants in diverse applications
- Implement stabilization techniques like gradient penalty, feature matching, and virtual batch normalization for robust GAN training

Generative Modeling

- The goal is to design parameterized densities with very large capacity
- **Normalizing flows** sequence of nonlinear transformations to a simple dist $\mathrm{p}_z(\boldsymbol{z})$

  $\mathrm{p}(\boldsymbol{x} \mid \theta_{0:k}) = \mathrm{p}_z(\boldsymbol{z})$ where $z = f_{\theta_k}^{-1} \circ \cdots \circ f_{\theta_1}^{-1} \circ f_{\theta_0}^{-1}(\boldsymbol{x})$

- $f_{\theta_j}^{-1}$ must be invertible with tractable log-determinant of Jacobians
- **VAE** latent-variable models where inference networks specify parameters

  $\mathrm{p}(\boldsymbol{x}, \boldsymbol{y} \mid \theta) = \mathrm{p}(\boldsymbol{x} \mid f_\theta(\boldsymbol{y}))\mathrm{p}_y(\boldsymbol{y})$

- The marginal likelihood can be maximized via the Evidence Lower BOund (**ELBO**) which is a variational lower bound
  - A tractable approximation to the intractable marginal likelihood of a dataset

Generative Adversarial Networks

- GAN is an independent **pair** of components: a generator G and a discriminator D
- Training process is a **minimax** game

$$\min_G \max_D V(D,G) = \min_G \max_D \mathbf{E}_{\boldsymbol{x}\sim \mathrm{P}_d}[\log D(\boldsymbol{x})] + \mathbf{E}_{\boldsymbol{z}\sim \mathrm{P}_z}[\log(1 - D(G(\boldsymbol{z})))]$$

- **Theoretical guarantee** this minimax game has a global optimum for $\mathrm{P}_g = \mathrm{P}_d$
  - Not an easy task for a GAN training
- P_d is the actual data distribution
- P_g is the generated data distribution
- P_z is the chosen prior in latent vector space
- D wants $D(\boldsymbol{x}) = 1$ and $D(G(\boldsymbol{z})) = 0$ and G wants $D(G(\boldsymbol{z})) = 1$

Manifold P

- The data distribution P_d is assumed to lie on a lower-dimensional manifold
  - Manifold P is the data distribution where real data points lie
- The support set of a distribution P is the subset of the sample space where the probability distribution is non-zero
  - $\mathrm{supp}(P)$ is the set of all points x such that $\mathrm{supp}(P) = \{x \in X \mid P(x) > 0\}$
  - Support set of P is the actual set of points within that manifold where the distribution P assigns non-zero probability
- The generator G aims to map random noise to this manifold P, and the discriminator D tries to distinguish between real data points from P and the generated ones
- Optimizing a GAN involves training G to approximate P as closely as possible

How to Train a GAN

- D should maximize $\mathbf{E}_{\boldsymbol{x}\sim \mathrm{P}_d}[\log D(\boldsymbol{x})] + \mathbf{E}_{\boldsymbol{z}\sim \mathrm{P}_z}[\log(1 - D(G(\boldsymbol{z}))]$
- G should minimize $\mathbf{E}_{\boldsymbol{z}\sim \mathrm{P}_z}[\log(1 - D(G(\boldsymbol{z}))]$
- G can alternatively maximize $\mathbf{E}_{\boldsymbol{z}\sim \mathrm{P}_z}[\log(D(G(\boldsymbol{z}))]$
- Gradient descent or ascent (for maximization) to optimize parameters θ_D and θ_G
- Stability via gradient penalty $\lambda\, \mathbf{E}_{\hat{x}\sim \mathrm{P}_{\hat{x}}}[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2]$
- Learning rate scheduling as different learning rates for G and D, $\eta_G < \eta_D$
  - Empirically, slower learning rates for G and faster for D tend to help with stability
- Batch normalization on layer activations to stabilize training $x_{\mathrm{norm}} = \frac{x-\mu_B}{\sqrt{\sigma_B^2+\epsilon}}$
- Adam optimizer use parameters β_1 and β_2 to control the moment estimates

The Optimal Discriminator

$$f = \mathbf{E}_{\boldsymbol{x}\sim \mathrm{P}_d}[\log D(\boldsymbol{x})] + \mathbf{E}_{\boldsymbol{z}\sim \mathrm{P}_z}[\log(1 - D(G(\boldsymbol{z}))]$$

$$f = \int_x [\mathrm{P}_d(x)\log D(x) + \mathrm{P}_g(x)\log(1 - D(x))]dx$$

$$\frac{\partial f}{\partial D(x)} = \frac{\mathrm{P}_d(x)}{D(x)} - \frac{\mathrm{P}_g(x)}{1 - D(x)} = 0$$

$$\big(1 - D(x)\big)\mathrm{P}_d(x) = D(x)\mathrm{P}_g(x)$$

$$D^*(x) = \frac{\mathrm{P}_d(x)}{\mathrm{P}_g(x) + \mathrm{P}_d(x)}$$

- Assumption, maximizing the **integrands** for all $\boldsymbol{x}$ is sufficient
  - The integrand f is **concave** for $D(\boldsymbol{x}) \in (0,1)$

The Optimal Generator

- Given an optimal discriminator D^*

$$f = \mathbf{E}_{\boldsymbol{x}\sim \mathrm{P}_d}[\log D(\boldsymbol{x})] + \mathbf{E}_{\boldsymbol{z}\sim \mathrm{P}_g}[\log(1 - D(\boldsymbol{x})]$$

$$f = \mathbf{E}_{\mathrm{P}_d} \log \frac{\mathrm{P}_d(x)}{\mathrm{P}_g(x) + \mathrm{P}_d(x)} + \mathbf{E}_{\mathrm{P}_g} \log \frac{\mathrm{P}_g(x)}{\mathrm{P}_g(x) + \mathrm{P}_d(x)}$$

$$\propto \frac{1}{2}\mathrm{KL}\left(\mathrm{P}_d \parallel \frac{\mathrm{P}_g + \mathrm{P}_d}{2}\right) + \frac{1}{2}\mathrm{KL}\left(\mathrm{P}_g \parallel \frac{\mathrm{P}_g + \mathrm{P}_d}{2}\right)$$

- Above term is Jensen-Shannon Divergence (JSD)
- f achieves its global minimum at $\mathrm{P}_g = \mathrm{P}_d$
- The generator wants to minimize $\mathrm{JSD}(\mathrm{P}_d \parallel \mathrm{P}_g) - \log 4$

Kullback-Leibler Divergence

- A measure of how one probability distrib Q diverges from a reference distribution P
- Quantifies the difference between the two distributions
  - Measuring how much information is lost when approximating P with Q

$$\mathrm{KL}(P \parallel Q) = \sum_{x} P(x) \log \frac{P(x)}{Q(x)}$$

- KL divergence tells how inefficient it is to use distribution Q to represent P
- Not symmetric and non-negative, can be used for any density
- A zero indicates that P and Q are identical

Fundamental Problems

- There is no static optimal discriminator
- D indicates the direction for G should move relative to the current generator
- For a given fixed D, the optimal generator outputs $\operatorname{argmax} D(\boldsymbol{x}) \; \forall \; z \sim Z$
- Cannot train generator without training discriminator first
- **Optimization Issues**
  - Simultaneous updates require a careful balance between players
  - Stationary point exists but there is no guarantee of reaching it
  - If D is undertrained, it guides the generator in the wrong direction
  - If D is overtrained, it may be tough for G to make a progress

Fundamental Problems

- **Vanishing Gradients** the D becomes "too good" and the G gradient vanishes
- **Non-Convergence** the G and D oscillate without reaching an equilibrium
- **Mode Collapse** the G distribution collapses to a small set of examples
- **Mode Dropping** the G distribution does not fully cover the data distribution
- Some remedies of distance measures
  - VAE **Kullback-Leibler Divergence**
  - GAN **Jenson-Shanon Divergence**
  - WGAN **Wasserstein Distance** (Kantorovich-Rubinstein duality used)

Vanishing Gradients

- **Change Objectives** Use non-saturating heuristic objective, maximum-likelihood cost, etc.
- **Limit Discriminator** Restrict the capacity of the discriminator
- **Schedule Learning** Try to balance training D_{θ_d} and G_{θ_g} by parameterization
  - Highly non-convex
- **Non-Convergence** due to the training tends to oscillate
- **Hacks**
  - Use soft and noisy labels
  - If DCGAN will not work, try KL-GAN or VAE-GAN
  - Use reinforcement learning methods, replay buffer, deterministic policy gradients
  - Use optimizer SGD for D and Adam for G

Mode Collapse and Mode Dropping

- Stochastic Gradient Descent may optimize the max-min objective
- **Intuition** the G maps all $\boldsymbol{z}$ values to the $\hat{\boldsymbol{x}}$ that is mostly likely to confuse the D
- Use alternative divergences, such as **f-Divergences**

$$\mathrm{D}_f(P \parallel Q) = \int_x P(x) f\left(\frac{P(x)}{Q(x)}\right) dx$$

- Use **Integral Probability Metrics**, e.g., Earth Movers Distance, Maximum Mean Discrepancy

$$\gamma_f(P \parallel Q) = \sup_{f \in F} \left| \int f dP - \int f dQ \right|$$

- **Wasserstein GAN** avoids mode collapse better

Feature Matching

- Statistics of generated images should match the statistics of real images
- D produces multidimensional output, a **statistic** of the data
- G trained to minimize the L_2 error between real and generated data
- D trained to maximize L_2 between real and generated data

$$\mathrm{L}_{\mathrm{FM}} = \left\| \mathbf{E}_x D(\boldsymbol{x}) - \mathbf{E}_z D\big(G(\boldsymbol{z})\big) \right\|_2^2$$

Minibatch Discrimination

- Discriminator looks at multiple inputs at once and decide if those inputs come from the real or generated distribution
- GANs frequently collapse to a single point
- Discriminator needs to differentiate between two distributions
  - Encouraging the G to produce a variety of outputs
  - Measure of diversity

Historical Averaging

- Reduce oscillations by updates converge to a mean
- Add a regularization term to decrease oscillations to both the G and D loss

$$\lambda \cdot \sum_{i} \left(\theta_i - \frac{1}{t} \sum_{k=1}^{t} \theta_i^k \right)^2$$

  - θ_i are the model parameters
  - θ_i^k are the past iterations
  - λ is the regularization parameter

One-Sided Label Smoothing

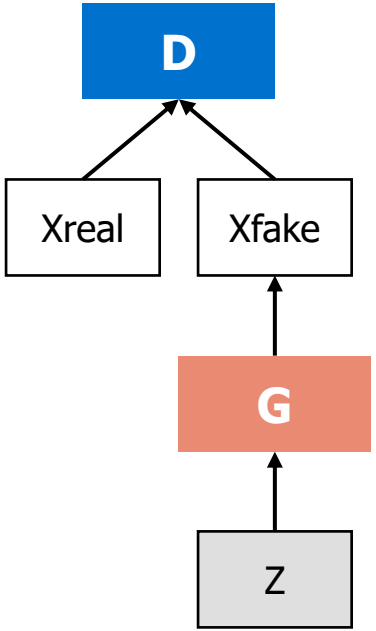
- Avoid overpenalizing generated images
  - Sigmoid might saturate to 0 or 1 not reaching the goal
  - Provide targets that are ϵ or $1 - \epsilon$ so the sigmoid does not saturate
- Experimentally smooth real targets but do not smooth generated targets when training the D

Virtual Batch Normalization

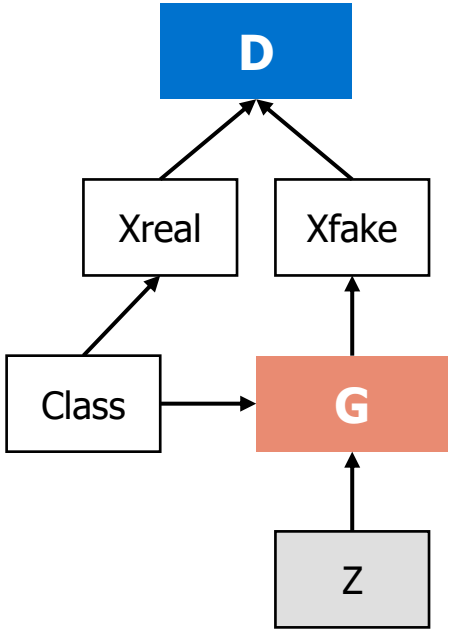
- Batch normalization can speed up convergence
- May not be the best method in adversarial learning
- Collect statistics on a fixed batch of real data and use it to normalize others
  - Can reduce oscillations
  - More stable feature normalization

GAN Architectures

- Generative-discriminative pair

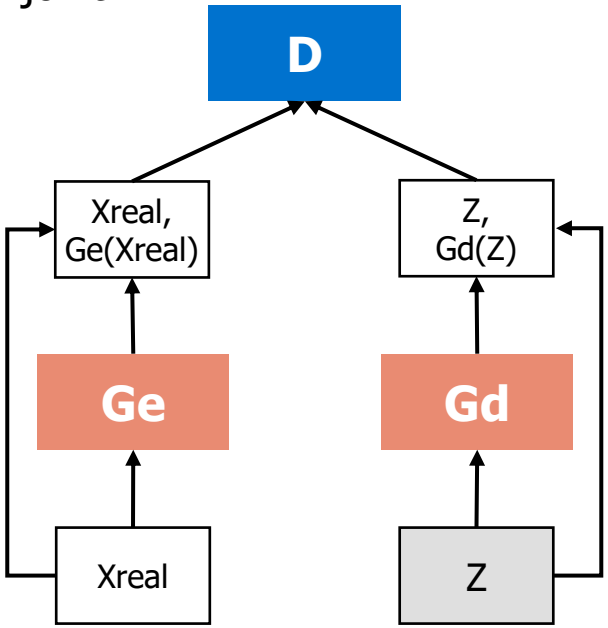


- Conditional GAN

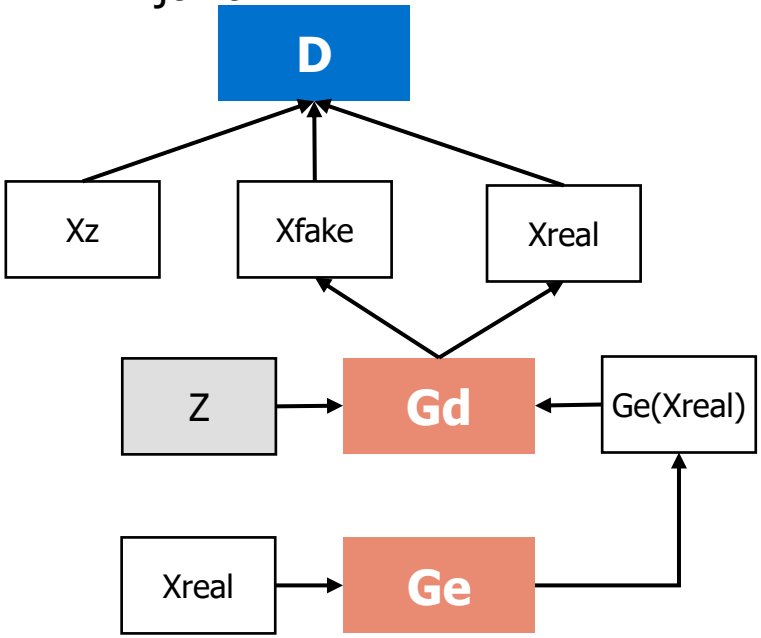


GAN Architectures

- Latent space autoencoder Ge and Gd, joint

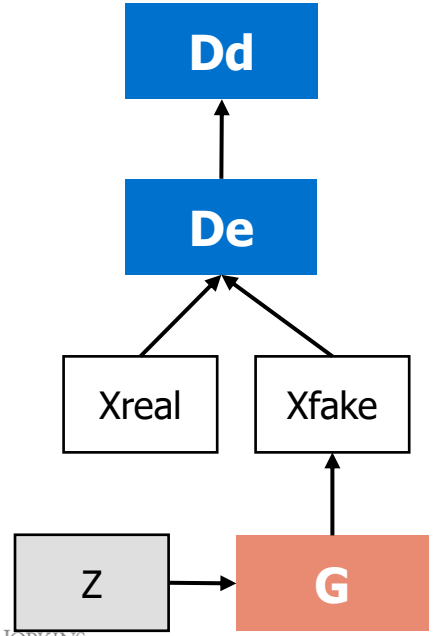


- Data space autoencoder Ge and Gd, joint

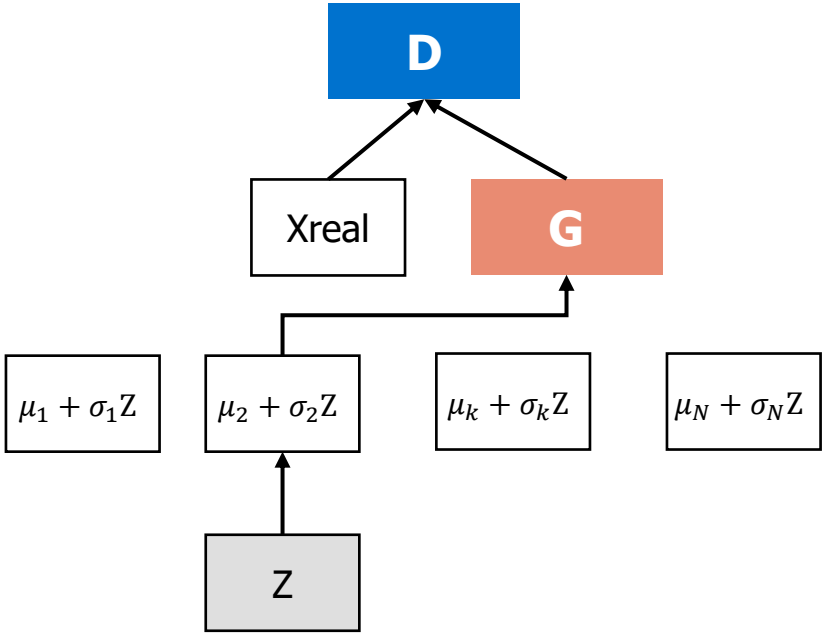


GAN Architectures

- Improved discriminator D encoder, D decoder



- Latent space engineering



Conditional GAN

- **Input Conditioning** cGANs have both the generator $G(z|y)$ and discriminator $D(x|y)$ conditioned on an additional variable y (e.g., labels)

$$\min_G \max_D \mathbf{E}_{\boldsymbol{x}\sim \mathrm{P}_d(x|y)}\left[\log D(\boldsymbol{x}|\boldsymbol{y})\right] + \mathbf{E}_{\boldsymbol{z}\sim \mathrm{P}_z(z)}\left[\log\left(1 - D(G(\boldsymbol{z}|\boldsymbol{y})|\boldsymbol{y})\right)\right]$$

- **Discriminator Input** $D(x|y)$ takes both real/fake x and the condition y as inputs
- **Generator Objective** G must generate samples $G(z|y)$ that not only look real but also correspond to the condition y
- **Model Expressiveness** cGANs enable control over generated samples via y
- Improved applications include image-to-image translation, class-conditional generation, data augmentation, supervised learning enhancement

InfoGAN

- **Latent Code** InfoGAN introduces an additional latent code c to capture interpretable and meaningful features, modifying the generator to $G(z, c)$
- **Mutual Information Objective** maximizes the mutual information $\boldsymbol{I}(c; G(z, c))$ between the latent code c and the $G(z, c)$ to encourage interpretability of c

$$\min_{G} \max_{D} V(D, G) - \lambda \boldsymbol{I}(c; G(z, c))$$

- **Auxiliary Network Q** introduces an auxiliary network $Q(c|x)$ to approximate the posterior $p(c|x)$ to maximize mutual information which trained jointly with G and D
- **Disentangled Representation** InfoGAN aims to disentangle the representation by learning interpretable latent factors c
- Improved applications include disentangled representation learning, unsupervised feature discovery, generative data manipulation, improving generative models for interpretable outputs

Wasserstein GAN

- **Wasserstein Distance** WGAN uses the Wasserstein (or Earth Mover's) distance
  - More stable gradients and helps avoid vanishing gradients
  - Directly measures how much "work" it takes to transform one dist into another
    - Unlike JSD, which focuses only on whether two distributions overlap
- **Lipschitz Constraint** used a 1-Lipschitz continuity condition on the critic (D)
  - Initially used weight clipping, later improved with gradient penalty in WGAN-GP
    - Limit the weights to the range $[-0.01, 0.01]$
  - A function f is Lipschitz continuous if there exists a constant K such that for any two points x_1 and x_2, $|f(x_1) - f(x_2)| \leq K \cdot |x_1 - x_2|$ is satisfied
  - D must satisfy Lipschitz continuity condition, so D training is more stable
- **Improved Training Stability** achieved by W-distance and a well-regularized critic

CycleGAN

- **Two Generators** CycleGAN introduces two generators, $G: X \to Y$ and $F: Y \to X$, to learn mappings between two domains X and Y
- **Cycle Consistency Loss** introduces a cycle consistency loss to ensure that the mappings are invertible

$$L_{cyc}(G,F) = \mathbf{E}_{x\sim p_X(x)}\left[\left\|F\big(G(x)\big) - x\right\|_1\right] + \mathbf{E}_{y\sim p_Y(y)}\left[\left\|G\big(F(y)\big) - y\right\|_1\right]$$

- **Two Discriminators** uses D_X and D_Y to distinguish between real and generated images in domain X and Y, respectively
- **Unpaired Data** CycleGAN is designed to work with unpaired data from X and Y, learning mappings without explicit correspondence between samples
- Improved applications include unpaired image-to-image translation, style transfer, image restoration, domain adaptation (synthetic vs real)

StyleGAN

- **Mapping Network** StyleGAN uses a mapping network $f: z \rightarrow w$ transforms the z into an intermediate latent space w, modulates the G as $G(w) = G(f(z))$
- **Style Modulation** each layer of the generator is modulated by the latent code w via adaptive instance normalization

$$\mathrm{AdaIN}(x, w) = \gamma(w)\left(\frac{x - \mu(x)}{\sigma(x)}\right) + \beta(w)$$

- **Stochastic Variation** introduces stochastic variation by adding noise to intermediate layers, allowing fine details to vary independently of the latent code w
- **Progressive Growing** StyleGAN employs a progressive growing technique, starting from low-resolution images and incrementally increasing the resolution in training
- Improved applications include high-resolution image synthesis, face generation and editing, art and design content creation, 3D-aware image synthesis (StyleGAN3)

References

- Foster, David. Generative deep learning. " O'Reilly Media, Inc.", 2023.
- Zhu, Jun-Yan, et al. "Unpaired image-to-image translation using cycle-consistent adversarial networks." Proceedings of the IEEE international conference on computer vision. 2017. (cited 23k as of 2024)
- Karras, Tero, Samuli Laine, and Timo Aila. "A style-based generator architecture for generative adversarial networks." Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2019. (cited 11k as of 2024)
- Chen, Xi, et al. "Infogan: Interpretable representation learning by information maximizing generative adversarial nets." Advances in neural information processing systems 29 (2016). (cited 5k as of 2024)
- Li, Yanchun, et al. "The theoretical research of generative adversarial networks: an overview." Neurocomputing 435 (2021): 26-41.

JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

© The Johns Hopkins University 2025, All Rights Reserved.